

Artificial Neural Network

Kritanta Saha

Assistant Professor
Dept. of Computer Science & Engineering
Sister Nivedita University, Kolkata.

Problem with Logistic Regression

- Logistic regression is a supervised algorithm for classification that predicts probabilities of the occurrence of an event. For non-linear classification, Use Sigmoid function.
- However, applicable for problems with less number of features.
- If the feature size n is too large then there are $O(n^2)$ quadratic features and $O(n^3)$ number of cubic features.

$$g(\Theta_0 + \Theta_1 x_1 + \Theta_2 x_2 + \Theta_3 x_1 x_2 + \Theta_4 x_1^2 x_2 + \Theta_5 x_1^3 x_2 + \Theta_6 x_1 x_2^2 + \dots)$$

- Training such a model becomes expensive.

Introduction to Artificial Neural Network

- An *Artificial Neural Network(ANN)* is a model of computation inspired by *the structure of neural networks in the brain*.
- Brain consists of a large number of neurons.
- Neurons are the basic computing devices of brains.

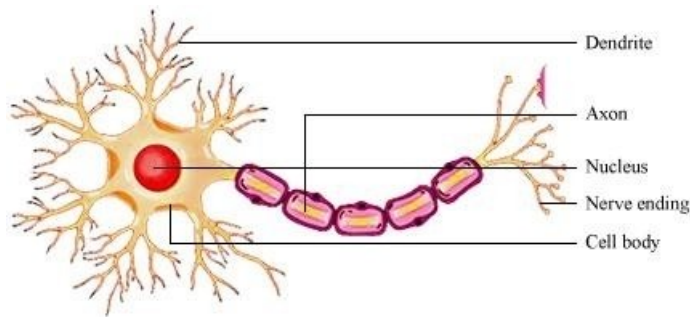


Figure 1: Neuron

Neural Network

- Neurons are connected in a complex communication network, through which the brain can perform highly complex computations.

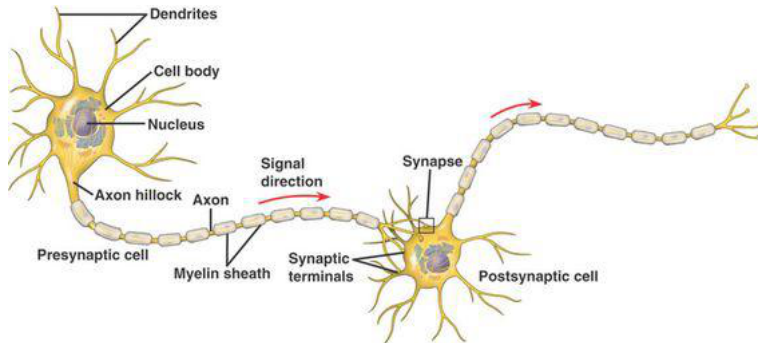
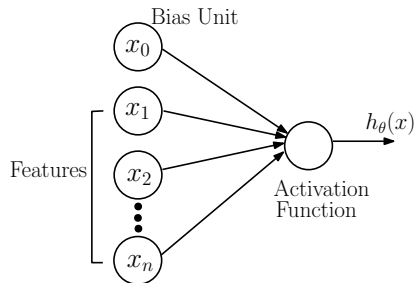
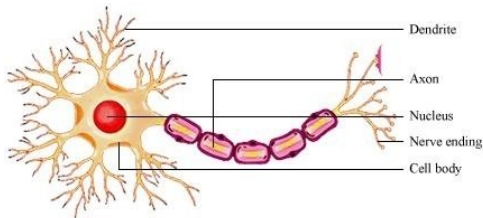


Figure 2: Neural Network

- ANN are formal computation constructs modeled after this computation paradigm.

Model Representation



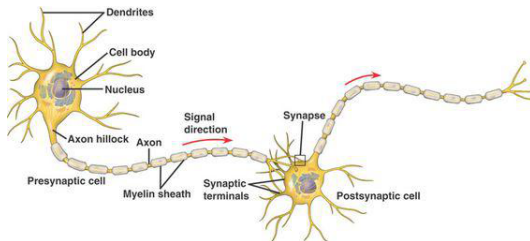
$$X = \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ \vdots \\ x_n \end{bmatrix} \in \mathbb{R}^{n+1} \quad \Theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \vdots \\ \theta_n \end{bmatrix} \in \mathbb{R}^{n+1}$$

- Activation Function: Sigmoid/Logistic function $g(z)$

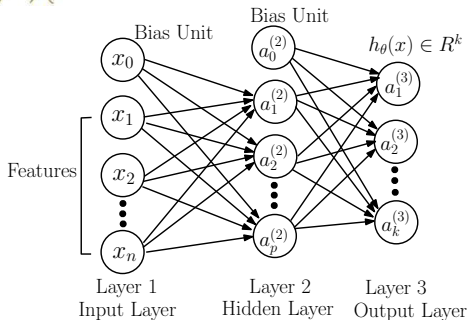
$$g(z) = \frac{1}{1 + e^{-z}}$$
$$h_\theta(x) = g(\Theta^T x) = \frac{1}{1 + e^{-\Theta^T x}}$$

where $0 \leq h_\theta(x) \leq 1$

Model Representation



- ANN can be described as a directed graph whose nodes corresponds to neurons and edges corresponds to links between them.



McCulloch Pitts (MP) Neuron

- $x_1, x_2, \dots, x_n \in \{0, 1\}$
- $y \in \{0, 1\}$
- g : aggregates the input
- f : takes a based on this aggregation

$$g(x_1, x_2, \dots, x_n) = g(X) = \sum_{i=1}^n x_i$$

$$y = f(g(X)) = 1 \text{ if } g(x) \geq \theta$$

$$= 0 \text{ if } g(x) < \theta$$

where θ is a thresholding parameter.

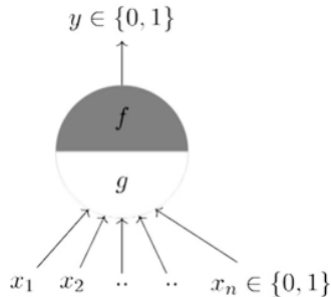


Figure 3: MP neurons

Implementation of Boolean Function using MP Neurons

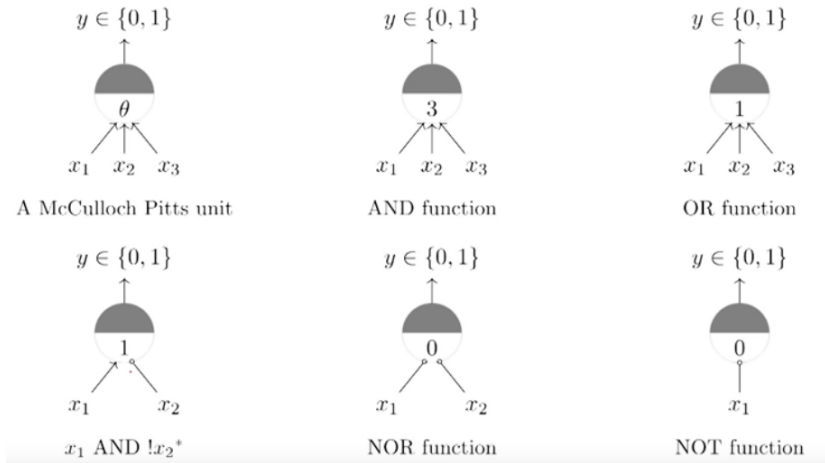
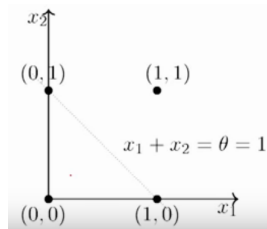
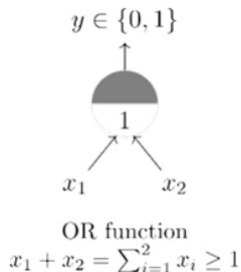


Figure 4: Boolean functions using MP neurons

Geometric Interpretation of MP Neurons



- A single McCulloch Pitts neuron can be used to represent boolean functions that are linearly separable.
- what about non-boolean inputs?
- Do we always need to hand-pick the threshold?
- What if some inputs have more importance than others?
- What about functions that are not linearly separable?

Perceptrons

- $x_1, x_2, \dots, x_n \in \mathbb{R}$
- Introduce numerical weights w_i for each input x_i .
- Mechanism for learning these weights.

$$y = 1 \text{ if } \sum_{i=1}^n w_i \times x_i \geq \theta$$
$$= 0 \text{ if } \sum_{i=1}^n w_i \times x_i < \theta$$

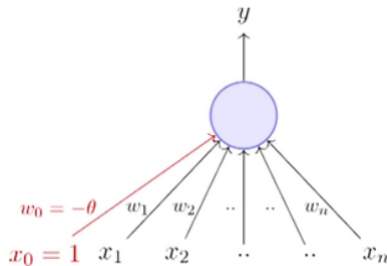


Figure 5: Perceptron

$$y = 1 \text{ if } \sum_{i=0}^n w_i \times x_i \geq 0$$
$$= 0 \text{ if } \sum_{i=0}^n w_i \times x_i < 0$$

Implementation of Boolean Function using Perceptrons

Implementation of OR operation

x_1	x_2	x_1+x_2	
0	0	0	$w_0 + \sum_{i=1}^2 w_i \times x_i < 0$
0	1	1	$w_0 + \sum_{i=1}^2 w_i \times x_i \geq 0$
1	0	1	$w_0 + \sum_{i=1}^2 w_i \times x_i \geq 0$
1	1	1	$w_0 + \sum_{i=1}^2 w_i \times x_i \geq 0$

$$w_0 + w_1.0 + w_2.0 < 0 \implies w_0 < 0$$

$$w_0 + w_1.0 + w_2.1 \geq 0 \implies w_2 \geq -w_0$$

$$w_0 + w_1.1 + w_2.0 \geq 0 \implies w_1 \geq -w_0$$

$$w_0 + w_1.1 + w_2.1 \geq 0 \implies w_1 + w_2 \geq -w_0$$

One possible solution $w_0 = -1$, $w_1 = 1.1$ and $w_2 = 1.1$

How Perceptron will learn to find such weights?

Perceptron Learning Algorithm

- Learning weights and threshold

- Consider two vectors W and X

$$W = [w_0, w_1, \dots, w_n] \quad X = [1, x_1, x_2, \dots, x_n]$$

$$\sum_{i=0}^n w_i \times x_i = W^T X$$

$$\begin{aligned} y &= 1 && \text{if } W^T X \geq 0 \\ &= 0 && \text{if } W^T X < 0 \end{aligned}$$

- $W^T X = 0$: Decision Boundary.
- $W^T X = W \cdot X \implies$ they are orthogonal.
- $\cos \alpha = \frac{W^T X}{\|W\| \|X\|} = 0$

- $y = 1$

$$\begin{aligned} \text{if } W^T X \geq 0 &\implies \cos \alpha \geq 0 \\ &\implies \alpha \leq 90 \end{aligned}$$

- $y = 0$

$$\begin{aligned} \text{if } W^T X < 0 &\implies \cos \alpha < 0 \\ &\implies \alpha > 90 \end{aligned}$$

where α : Angle between X and current W .

Perceptron Learning Algorithm

Consider Actual class label is 1

- If Predicted Class label is 0
i.e., if $W \cdot X < 0 \implies \alpha > 90$
- As Actual label is 1, we want $\alpha < 90$.
- Update W i.e., $W_{new} = W + X$
- Let α_{new} be the angle between W_{new} and X , then

$$\begin{aligned}\cos(\alpha_{new}) &\propto W_{new}^T X \\ &\propto (W + X)^T X \\ &\propto W^T X + X^T X \\ &\propto \cos\alpha + X^T X\end{aligned}$$

$$\implies \cos(\alpha_{new}) > \cos\alpha$$

$$\implies \alpha_{new} < \alpha$$

Consider Actual class label is 0

- If predicated Class label is 1
i.e., if $W \cdot X \geq 0 \implies \alpha < 90$
- As Actual label is 0, we want $\alpha > 90$.
- Update W i.e., $W_{new} = W - X$
- Let α_{new} be the angle between W_{new} and X , then

$$\begin{aligned}\cos(\alpha_{new}) &\propto W_{new}^T X \\ &\propto (W - X)^T X \\ &\propto W^T X - X^T X \\ &\propto \cos\alpha - X^T X\end{aligned}$$

$$\implies \cos(\alpha_{new}) < \cos\alpha$$

$$\implies \alpha_{new} > \alpha$$

Perceptron Learning Algorithm

Algorithm 1: Perceptron Learning Algorithm

P =Inputs with label 1;

N =Inputs with label 0;

Initialize $W = [w_0, w_1, \dots, w_n]$ randomly;

while *not convergence* **do**

 Pick random $X \in P \cup N$;

if $X \in P$ and $\sum_{i=0}^n w_i \times x_i < 0$ **then**

$W = W + X$;

end

if $X \in N$ and $\sum_{i=0}^n w_i \times x_i \geq 0$ **then**

$W = W - X$;

end

end

The algorithm converges when all the inputs are classified correctly.

Implement XOR using a single Perceptron

x_1	x_2	XOR	
0	0	0	$w_0 + \sum_{i=1}^2 w_i \times x_i < 0$
0	1	1	$w_0 + \sum_{i=1}^2 w_i \times x_i \geq 0$
1	0	1	$w_0 + \sum_{i=1}^2 w_i \times x_i \geq 0$
1	1	0	$w_0 + \sum_{i=1}^2 w_i \times x_i < 0$

$$w_0 + w_1.0 + w_2.0 < 0 \implies w_0 < 0$$

$$w_0 + w_1.0 + w_2.1 \geq 0 \implies w_2 \geq -w_0$$

$$w_0 + w_1.1 + w_2.0 \geq 0 \implies w_1 \geq -w_0$$

$$w_0 + w_1.1 + w_2.1 \geq 0 \implies w_1 + w_2 < -w_0$$

We have a contradiction.

- A single perceptron can not deal with data not linearly separable.
- We need a network of perceptrons to deal with such data.

Network of Perceptrons

Consider True=+1 and
False=-1.

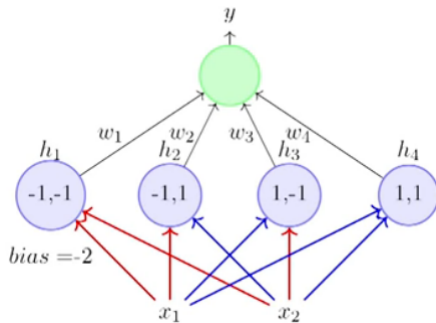
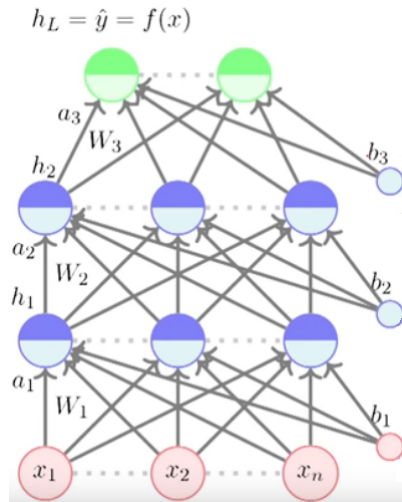


Figure 6: XOR using network of perceptrons

x_1	x_2	XOR	h_1	h_2	h_3	h_4	
0	0	0	1	0	0	0	$w_0 + \sum_{i=1}^4 w_i \times h_i < 0$
0	1	1	0	1	0	0	$w_0 + \sum_{i=1}^4 w_i \times h_i \geq 0$
1	0	1	0	0	1	0	$w_0 + \sum_{i=1}^4 w_i \times h_i \geq 0$
1	1	0	0	0	0	1	$w_0 + \sum_{i=1}^4 w_i \times h_i < 0$

Feedforward Neural Network

- Input: n dimensional vector.
- $L - 1$ hidden layers having n neurons each.
- One output layer containing k neurons.
- Each Neuron consists of
 - Pre-activation function:
 - For layer i : $a_i(x) = b_i + W_i h_{i-1}(x)$
 - Activation function:
 - for layer i : $h_i(x) = g(a_i(x))$
 - for output layer: $h_L(x) = f(x) = \hat{y} = O(a_L(x))$
- $W_i \in \mathbb{R}^{n \times n}$ and $b_i \in \mathbb{R}^n$ between layers $i - 1$ and i where $0 < i < L$.
- $W_L \in \mathbb{R}^{n \times k}$ and $b_k \in \mathbb{R}^k$ between the last hidden layer and the output layer.



Feedforward Neural Network (cont..)

- The Pre-activation at layer i : $a_i = b_i + W_i h_{i-1}$
- The Activation at layer i : $h_i = g(a_i)$ where g is called the activation function. (Eg. logistic, tanh, linear etc).
- The Activation of output layer : $h_L = f = \hat{y} = O(a_L)$ where O is the output activation function. (Eg. Softmax, linear etc.)
- Data : $\{x_i, y_i\}_{i=1}^N$
- Model : $\hat{y}_i = f(x_i) = O(W_3 g(W_2 g(W_1 x + b_1) + b_2) + b_3)$
- Parameters : $\Theta = W_1, \dots, W_L, b_1, b_2, \dots, b_L$
- Algorithm : Gradient Descent with Backpropagation
- Objective/Loss/Error function:

$$\min \frac{1}{N} \sum_{i=1}^N \sum_{j=1}^k (\hat{y}_{ij} - y_{ij})^2$$

$$\min \mathbb{L}(\Theta)$$

ANN Learning: Gradient Descent Algorithm with Backpropagation

Algorithm 2: *Gradient_Descent()*

$t = 1$;

$max_iteration = 1000$;

Initialize $\Theta_0 = [w_1^0, \dots, w_L^0, b_1^0, \dots, b_L^0]$;

while $t < max_iteration$ **do**

$h_1, h_2, \dots, h_{L-1}, a_1, a_2, \dots, a_L, \hat{y} = Forward_Propagation(\Theta_t)$;

$\nabla \Theta_t = Back_Propagation(h_1, h_2, \dots, h_{L-1}, a_1, a_2, \dots, a_L, \hat{y})$;

$\Theta_{t+1} = \Theta_t - \eta \nabla \Theta_t$;

$t = t + 1$;

end

where $\nabla \Theta_t = \left[\frac{\partial \mathcal{L}(\Theta)}{\partial W_t}, \frac{\partial \mathcal{L}(\Theta)}{\partial b_t} \right]^T$

- $\nabla \Theta$ Composed of

$\nabla W_1, \nabla W_2, \dots, \nabla W_{L-1} \in \mathbb{R}^{n \times n}$ and $\nabla W_L \in \mathbb{R}^{n \times k}$

$\nabla b_1, \nabla b_2, \dots, \nabla b_{L-1} \in \mathbb{R}^n$ and $\nabla b_L \in \mathbb{R}^k$

Forward Propagation

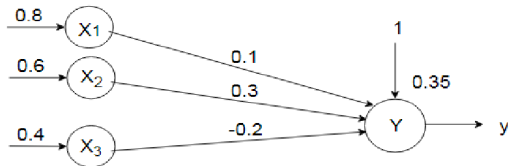
Algorithm 3: *ForwardPropagation*(Θ)

```
 $k = 1$  ;  
while  $k \leq L - 1$  do  
     $a_k = b_k + w_k h_{k-1}$  ;  
     $h_k = g(a_k)$  ;  
     $k = k + 1$  ;  
end  
 $a_L = b_L + w_L h_{L-1}$  ;  
 $\hat{y} = O(a_L)$  ;
```

Forward Propagation example 1:

Problem: (to compute Y out): apply activation function

- Calculate the output ' y ' of a 3 input neuron with bias, with data as given in the diagram. Use sigmoidal activation function



Forward Propagation example 1:

$$[x_1, x_2, x_3] = [0.8, 0.6, 0.4]$$

The weights are,

$$[w_1, w_2, w_3] = [0.1, 0.3, -0.2]$$

The net input can be calculated as,

$$y_{in} = b + \sum_{i=1}^n (x_i w_i)$$

$$y_{in} = 0.35 + 0.8 \times 0.1 + 0.6 \times 0.3 + 0.4 \times (-0.2)$$

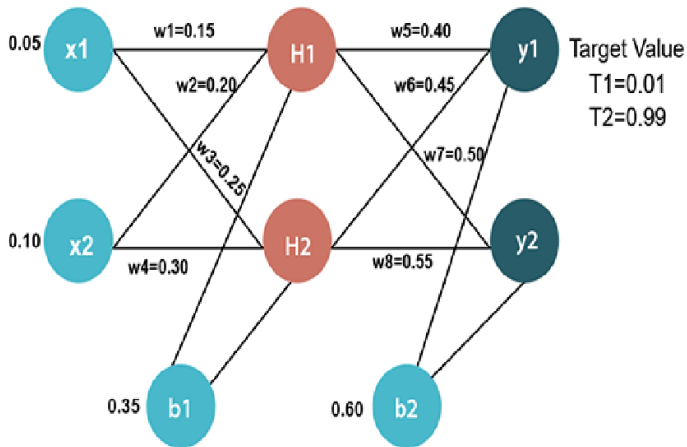
$$y_{in} = 0.35 + 0.08 + 0.18 - 0.08 = 0.53$$

(i) For Binary Sigmoidal Function,

$$y = f(y_{in}) = \frac{1}{1 + e^{-y_{in}}} = \frac{1}{1 + e^{-0.53}} = 0.62$$

Forward Propagation example 2:

Calculate the values of the preactivation and activation function by considering sigmoid as activation function for all the neurons:

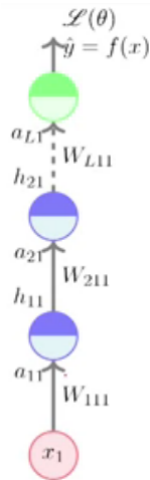


Backpropagation

- How to compute $\nabla\Theta$ which is Composed of
 $\nabla W_1, \nabla W_2, \dots, \nabla W_{L-1} \in \mathbb{R}^{n \times n}$ and $\nabla W_L \in \mathbb{R}^{n \times k}$
 $\nabla b_1, \nabla b_2, \dots, \nabla b_{L-1} \in \mathbb{R}^n$ and $\nabla b_L \in \mathbb{R}^k$

$$\frac{\partial \mathcal{L}(\Theta)}{\partial W_{111}} = \frac{\partial \mathcal{L}(\Theta)}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial a_{L11}} \cdot \frac{\partial a_{L11}}{\partial h_{21}} \cdot \frac{\partial h_{21}}{\partial a_{21}} \cdot \frac{\partial a_{21}}{\partial h_{11}} \cdot \frac{\partial h_{11}}{\partial a_{11}} \cdot \frac{\partial a_{11}}{\partial w_{111}}$$

$$\frac{\partial \mathcal{L}(\Theta)}{\partial W_{111}} = \frac{\partial \mathcal{L}(\Theta)}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial \mathbf{a}_3} \cdot \frac{\partial \mathbf{a}_3}{\partial h_2} \cdot \frac{\partial h_2}{\partial a_2} \cdot \frac{\partial a_2}{\partial h_1} \cdot \frac{\partial h_1}{\partial a_1} \cdot \frac{\partial a_1}{\partial w_{111}}$$



Backpropagation : Computing Gradients w.r.t the output unit

$$\frac{\partial \mathbb{L}(\Theta)}{\partial W_{111}} = \frac{\partial \mathbb{L}(\Theta)}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial \mathbf{a}_3} \cdot \frac{\partial \mathbf{a}_3}{\partial h_2} \cdot \frac{\partial h_2}{\partial a_2} \cdot \frac{\partial a_2}{\partial h_1} \cdot \frac{\partial h_1}{\partial a_1} \cdot \frac{\partial a_1}{\partial w_{111}}$$

$$\frac{\partial \mathbb{L}(\Theta)}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial a_{3i}} = \frac{\partial \mathbb{L}(\Theta)}{\partial a_{3i}}$$

In general,

$$\frac{\partial \mathbb{L}(\Theta)}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial a_{Li}} = \frac{\partial \mathbb{L}(\Theta)}{\partial a_{Li}}$$

We need to find $\nabla_{a_L} \mathbb{L}(\Theta)$.

$$\nabla_{a_L} \mathbb{L}(\Theta) = \begin{bmatrix} \frac{\partial \mathbb{L}(\theta)}{\partial a_{L_1}} \\ \frac{\partial \mathbb{L}(\theta)}{\partial a_{L_2}} \\ \vdots \\ \frac{\partial \mathbb{L}(\theta)}{\partial a_{L_k}} \end{bmatrix}$$

Backpropagation : Computing Gradients w.r.t Hidden units

$$\frac{\partial \mathbb{L}(\Theta)}{\partial W_{111}} = \frac{\partial \mathbb{L}(\Theta)}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial a_3} \cdot \frac{\partial a_3}{\partial \mathbf{h}_2} \cdot \frac{\partial h_2}{\partial \mathbf{a}_2} \cdot \frac{\partial a_2}{\partial \mathbf{h}_1} \cdot \frac{\partial h_1}{\partial \mathbf{a}_1} \cdot \frac{\partial a_1}{\partial w_{111}}$$

$$\frac{\partial \mathbb{L}(\Theta)}{\partial W_{111}} = \frac{\partial \mathbb{L}(\Theta)}{\partial \mathbf{h}_2} \cdot \frac{\partial h_2}{\partial \mathbf{a}_2} \cdot \frac{\partial a_2}{\partial \mathbf{h}_1} \cdot \frac{\partial h_1}{\partial \mathbf{a}_1} \cdot \frac{\partial a_1}{\partial w_{111}}$$

- We need to find $\nabla_{h_k} \mathbb{L}(\Theta)$ and $\nabla_{a_i} \mathbb{L}(\Theta)$ where $i \leq L - 1$

$$\nabla_{h_i} \mathbb{L}(\Theta) = \begin{bmatrix} \frac{\partial \mathbb{L}(\theta)}{\partial h_{i_1}} \\ \frac{\partial \mathbb{L}(\theta)}{\partial h_{i_2}} \\ \vdots \\ \frac{\partial \mathbb{L}(\theta)}{\partial h_{i_n}} \end{bmatrix}$$

$$\nabla_{a_i} \mathbb{L}(\Theta) = \begin{bmatrix} \frac{\partial \mathbb{L}(\theta)}{\partial a_{i_1}} \\ \frac{\partial \mathbb{L}(\theta)}{\partial a_{i_2}} \\ \vdots \\ \frac{\partial \mathbb{L}(\theta)}{\partial a_{i_n}} \end{bmatrix}$$

Backpropagation : Computing Gradients w.r.t Parameters

$$\frac{\partial \mathbb{L}(\Theta)}{\partial W_{111}} = \frac{\partial \mathbb{L}(\Theta)}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial a_3} \cdot \frac{\partial a_3}{\partial h_2} \cdot \frac{\partial h_2}{\partial a_2} \cdot \frac{\partial a_2}{\partial h_1} \cdot \frac{\partial h_1}{\partial a_1} \cdot \frac{\partial a_1}{\partial w_{111}}$$

- We need to find $\nabla_{w_k} \mathbb{L}(\Theta)$ and $\nabla_{b_k} \mathbb{L}(\Theta)$.

$$\nabla_{w_k} \mathbb{L}(\Theta) = \begin{bmatrix} \frac{\partial \mathbb{L}(\theta)}{\partial w_{k00}} & \frac{\partial \mathbb{L}(\theta)}{\partial w_{k01}} & \cdots & \frac{\partial \mathbb{L}(\theta)}{\partial w_{k0n-1}} \\ \frac{\partial \mathbb{L}(\theta)}{\partial w_{k10}} & \frac{\partial \mathbb{L}(\theta)}{\partial w_{k11}} & \cdots & \frac{\partial \mathbb{L}(\theta)}{\partial w_{k1n-1}} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial \mathbb{L}(\theta)}{\partial w_{kn-10}} & \frac{\partial \mathbb{L}(\theta)}{\partial w_{kn-11}} & \cdots & \frac{\partial \mathbb{L}(\theta)}{\partial w_{kn-1n-1}} \end{bmatrix}$$

$$\nabla_{b_i} \mathbb{L}(\Theta) = \begin{bmatrix} \frac{\partial \mathbb{L}(\theta)}{\partial b_{k0}} \\ \frac{\partial \mathbb{L}(\theta)}{\partial b_{k1}} \\ \vdots \\ \frac{\partial \mathbb{L}(\theta)}{\partial b_{kn}} \end{bmatrix}$$

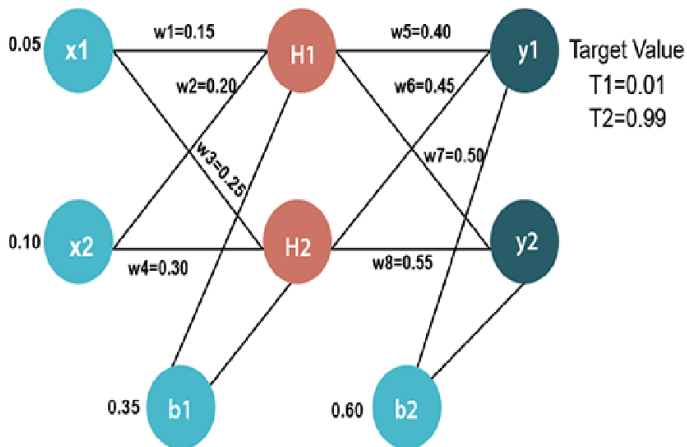
Backpropagation : Back Propagation Algorithm

Algorithm 4: *Back_Propagation*($h_1, h_2, \dots, h_{L-1}, a_1, a_2, \dots, a_L, \hat{y}$)

```
//Compute output Gradient ;  
 $\nabla_{a_L} \mathbb{L}(\Theta) = -(e(y) - f(x))$  ;  
 $k = L$  ;  
while  $k \geq 1$  do  
    //Compute gradient w.r.t parameters ;  
     $\nabla_{w_k} \mathbb{L}(\Theta) = \nabla_{a_k} \mathbb{L}(\Theta) h_{k-1}^T$  ;  
     $\nabla_{b_k} \mathbb{L}(\Theta) = \nabla_{a_k} \mathbb{L}(\Theta)$  ;  
    //Compute gradient w.r.t layer below ;  
     $\nabla_{h_{k-1}} \mathbb{L}(\Theta) = W_k^T (\nabla_{a_k} \mathbb{L}(\Theta))$  ;  
    //compute gradient w.r.t layer below ;  
     $\nabla_{a_{k-1}} \mathbb{L}(\Theta) = \nabla_{h_{k-1}} \mathbb{L}(\Theta) \odot [g'(a_{k-1,j})]$  ;  
     $k = k - 1$  ;  
end
```

Backpropagation example:

Calculate the weights:



Overview of Output and Loss functions used in ANN

Output and Loss functions for Regression Problem

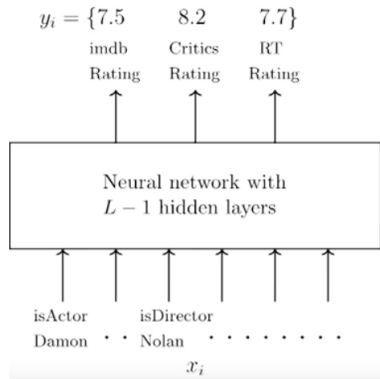
- How to choose the loss function $L(\Theta)$ for regression problem?

Example:

- If $y \in \mathbb{R}^n$ then
- Loss Function:

$$\mathbb{L}(\Theta) = \frac{1}{N} \sum_{i=1}^N \sum_{j=1}^3 (\hat{y}_{ij} - y_{ij})^2$$

- The output function O be
 $h_L = O(a_L) = W_O a_L + b_O$



Loss Function for Classification Problem

- For example Consider the following classification problem:
 - Classify [Apple, Banana, Cherry, Dragon fruit]
 - True Distribution : $p = [0, 1, 0, 0]$
 - Predicted Distribution : $q = [0.25, 0.45, 0.2, 0.1]$
- One way to defined loss function as

$$\sum_{i=1}^4 (p_i - q_i)^2$$

- Loss function considers the difference between true and predicted distribution.
- Cross Entropy can be used to find this difference between true and predicted distribution.

Entropy and Information Content

- Entropy: A measure of disorder.



Low

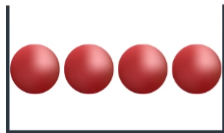


Medium



High

- Information Content :



- Information content measures the informativeness of an entity as the inverse of its probability of occurrence.

Relation Between Information content and Probability



$$IC(A) \propto \frac{1}{P(A)} \implies IC(A) = f(P(A))$$

- Let us consider two Independent random variable X and Y

$$IC(X \cap Y) = IC(X) + IC(Y)$$

$$f(P(X \cap Y)) = f(P(X) + f(P(Y))$$

$$f(P(X).P(Y)) = f(P(X)) + f(P(Y))$$

- Hence,

$$IC(X = A) = \log \left(\frac{1}{P(X = A)} \right) = -\log P(X = A)$$

Entropy

- Example: Let us a cricket tournament among teams A,B,C,D.
- True Result : $P = [0 \ 1 \ 0 \ 0]$ and profits are $V = [10k \ 20k \ 30k \ 40k]$
- Then the Expect: $\sum_{i=1}^4 P(i)V(i)$
- Now let us consider for a random variable X
- Probability distribution [$P(X=A)$ $P(X=B)$ $P(X=C)$ $P(X=D)$]
- Information contents [$IC(X=A)$ $IC(X=B)$ $IC(X=C)$ $IC(X=D)$]
- Entropy is the expected information content present in a Random variable.i.e.,

$$Entropy = \sum_{i \in A,B,C,D} P(X = i) IC(X = i)$$

$$Entropy = - \sum_{i \in A,B,C,D} P(X = i) \log P(X = i)$$

$$Entropy = - \sum_{i=1}^n p_i \log p_i$$

Cross Entropy

- Let the True Probability distribution $p = [p_1 \ p_2 \ p_3 \ p_4]$
- and the predicted distribution $q = [q_1 \ q_2 \ q_3 \ q_4]$
- $IC(X = i) = -\log q_i$
- The expected information content in the random variable is

$$-\sum_{i=1}^n p_i \log q_i$$

Called Cross Entropy.

- It captures the difference between the true and predicted probability distribution.

If $p = q$ then cross entropy is minimized.

$$\begin{aligned} & \text{Minimize}_{q_i} - \sum_{i=1}^n p_i \log q_i \\ & \text{s.t.} \quad \sum_{i=1}^n q_i = 1 \end{aligned}$$

Cross Entropy

If $p = q$ then cross entropy is minimized.

$$\begin{aligned} & \text{Minimize}_{q_i} - \sum_{i=1}^n p_i \log q_i \\ & \text{s.t. } \sum_{i=1}^n q_i = 1 \end{aligned}$$

or,

$$\text{Minimize}_{q_i} - \sum_{i=1}^n p_i \log q_i + \lambda(\sum q_i - 1)$$

For example,

$$\begin{aligned} \frac{\partial}{\partial q_2} - \sum_{i=1}^n p_i \log q_i + \lambda(\sum q_i - 1) &= 0 \\ \implies p_2 &= \lambda q_2 \end{aligned}$$

Output and Loss functions for Classification Problem

- Ensure that y and $\hat{y}$ are probability distribution.
- Pre-activation at output layer $a_L = b_L + W_L h_{L-1}$
- The output activation function $O : \hat{y}_j = O(a_L)_j = \frac{e^{a_{L,j}}}{\sum_{i=1}^k e^{a_{L,i}}}$ called Softmax function.
- where $O(a_L)_j$ is the j th element of $\hat{y}$ and $a_{L,j}$ is the j th element of the vector a_L .

After Ensuring both y and $\hat{y}$ are probability distribution now the question is how to capture there difference?

- Cross entropy can capture the difference : $\mathbb{L}(\theta) = -\sum_{c=1}^k y_c \log \hat{y}_c$
- $y_c = 1$ if $c = l$ (the true class label)
=0 otherwise
- $\mathbb{L}(\theta) = -\log \hat{y}_l$

Output and Loss functions for Classification Problem

- For classification problem, the objective function

$$\min_{\theta} \mathbb{L}(\Theta) = -\log \hat{y}_l$$

where $\hat{y}_l$ predicted probability of the correct event.

- $\hat{y}_l$ is a function of $\Theta = [W_1, W_2, \dots, W_L, b_1, b_2, \dots, b_L]$
- $\hat{y}_l = O[(W_3 g(W_2 g(W_1 x + b_1) + b_2) + b_3)]_l$

	Real Value Regression	Probability Classification
Output activation	linear	Softmax
Loss function	Squared Error	Cross Entropy